Session – 10

Task-1.

Create two tables: Influencers (id, name) and Collaborations (id, influencer1_id, influencer2_id, collab_date). Write a SQL FULL JOIN query to list all influencers and show their collaboration partner names if any, including influencers with no collaborations.

Solution:

```
 CREATE TABLE Influencers (
   id INT PRIMARY KEY,
   name VARCHAR(100)
);

CREATE TABLE Collaborations (
   id INT PRIMARY KEY,
   influencer1_id INT,
   influencer2_id INT,
   collab_date DATE
);

INSERT INTO Influencers (id, name)
VALUES
(1, 'Rahul'),
(2, 'Priya'),
(3, 'Amit'),
(4, 'Neha'),
(5, 'Riya');

INSERT INTO Collaborations
(id, influencer1_id, influencer2_id, collab_date)
VALUES
(1, 1, 2, '2026-10-01'),
(2, 2, 3, '2026-10-05'),
(3, 4, 5, '2026-10-10');
```

```
SELECT
  c.id AS collaboration_id,
  i1.name AS influencer1,
  i2.name AS influencer2,
  c.collab_date
FROM Collaborations c
FULL OUTER JOIN Influencers i1
  ON c.influencer1_id = i1.id
FULL OUTER JOIN Influencers i2
  ON c.influencer2_id = i2.id;
```

Task-2.

Using a SELF JOIN, write a query on a table called Playlists (id, user_id, playlist_name, parent_playlist_id) to display each playlist alongside its parent playlist name, similar to how Spotify shows nested playlists.

Hint: Join Playlists with itself on parent_playlist_id = id.

Solution:

```
SELECT
  child.playlist_name AS playlist,
  parent.playlist_name AS parent_playlist
FROM Playlists child
LEFT JOIN Playlists parent
  ON child.parent_playlist_id = parent.id;
```

Task-3.

Given three tables: Users (id, username), Orders (id, user_id, order_date), and Payments (id, order_id, amount), write a SQL query using multiple JOINs to display each username, their order date, and payment amount, showing all users even if they have no orders or payments.

Solution:

```
SELECT
  u.username,
  o.order_date,
```

```
    p.amount AS payment_amount
FROM Users u
LEFT JOIN Orders o
    ON u.id = o.user_id
LEFT JOIN Payments p
    ON o.id = p.order_id;
```

Task-4.

You notice that your JOIN query between Zomato's Restaurants and Reviews tables is returning duplicate rows for some restaurants. Modify your query to eliminate duplicates and explain in one line why the duplicates were happening.

Hint: Use DISTINCT or GROUP BY and consider the relationship between restaurants and reviews.

Solution:

```
SELECT DISTINCT
    r.id,
    r.name
FROM Restaurants r
JOIN Reviews rev
    ON r.id = rev.restaurant_id;
```

Task-5.

Write two different JOIN queries on a Products and Categories table (like Flipkart) to list all products with their category names, but use different join conditions in each. Briefly explain which join condition is more efficient and why.

Solution:

```
SELECT
    p.product_name,
    c.category_name
FROM Products p
JOIN Categories c
    ON p.category_id = c.id;
```